BrainAI

ProSuite Industrial

ZERO-TRUST COMMUNICATION

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 ÖFFENTLICHE KLASSIFIZIERUNG
Core Architecture & Development	**ProChat System-Handbuch**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie / Revision History

Version	Datum	Autor	Status	Beschreibung
1.0.0	01.01.2026	S. Köhne	Stable	Forward the Secret
2.0.0	26.02.2026	S. Köhne	Stable	High-Speed & Zero-Trust Edition (AEAD Header Protection, Noise-Padding, Strict Length Validation)

1 Produktübersicht

Die ProChat Core Engine v2.0.0 ist ein hochsicheres Kommunikationsmodul, das für den Einsatz in extremen Hochsicherheitsumgebungen (Zero-Trust) entwickelt wurde. Sie stellt die Vertraulichkeit, Integrität und Authentizität von Datenströmen sicher und schützt gegen moderne Angriffsszenarien wie Replay-Attacken, Traffic-Analysis und Man-in-the-Middle-Angriffe.

Kernmerkmale

- **Kryptografische Isolation (Ratchet):** Jede Session wird durch einen eigenen, asynchron rotierenden Ratchet-State isoliert.
- **True AEAD & Header Protection:** Sowohl die Nutzlast als auch die kritischen Metadaten (Sequenznummer, Typ, exakte Länge) werden in einem durchgehenden Cipher-Block verschlüsselt und authentifiziert.
- **ProKey Noise-Padding:** Leerer Paketraum wird nicht mit Nullen, sondern mit echter Hardware-Entropie (Rauschen) aufgefüllt, um Traffic-Analyse und Längen-Rückschlüsse durch Angreifer physikalisch unmöglich zu machen.
- **RAM-Vault Technologie:** Interner Schutz der aktiven Sitzungsschlüssel gegen Memory-Dumping zur Laufzeit.

2. Technische Spezifikationen

2.1 Globale Konstanten

Diese Werte sind für alle Implementierungen verbindlich und gewährleisten eine konstante Speicherkomplexität:

- **PC_PKT_SIZE:** *1024* Bytes (Feste Übertragungsgröße zur absoluten Verschleierung von Metadaten und Payload-Größen).
- **PC_MAX_PAYLOAD:** *988* Bytes (Maximal verfügbare Nutzlast pro Paket).
- **Key/Seed Size:** Exakt *32* Bytes (*256*-Bit Entropie).

2.2 Paket-Layout (Public Interface)

Ein ProChat-Paket ist nach außen hin ein vollständig opaker binärer Block von 1024 Bytes. Intern strukturiert es sich wie folgt:

- **Routing-Tag:** *4* Bytes (Klartext-Tag zur O(1)-Identifizierung der Session durch den Empfänger).
- **Encrypted Header & Payload:** *1004* Bytes (Vollständig verschlüsselter Bereich ab Byte 4. Enthält Absender-ID, TX-Counter, Nachrichtentyp, die exakte 16-Bit Payload-Länge sowie die eigentliche Nutzlast und das dynamische Entropie-Padding).
- **Auth-Tag (MAC):** *16* Bytes (Integritätssicherung des gesamten Pakets via ProHash Constant-Time HMAC).

3. Globales Error-Management

Das SDK verwendet ein hybrides Fehlermodell. Während die native Engine numerische Fehlercodes zurückgibt, transformieren die Wrapper diese in sprachspezifische Paradigmen (Exceptions oder Status-Objekte). Die Version 2.0.0 integriert harte Boundary-Checks direkt auf Core-Ebene.

Code	***Konstante***	***Beschreibung:***	***Kritikalität***
0	PC_OK	Operation erfolgreich abgeschlossen.	*Keine*
-1	PC_ERR_PARAM	Ungültige Pufferlängen, Oversized-Payloads oder Buffer-Overflow-Prävention.	*Mittel (Logikfehler)*
-2	PC_ERR_AUTH	MAC-Vergleich in konstanter Zeit fehlgeschlagen. Paket manipuliert!	*Hoch (Angriff!)*
-3	PC_ERR_REPLAY	Paket-Counter ungültig oder bereits verarbeitet.	*Mittel (Netzwerkfehler)*
-4	PC_ERR_DESYNC	Ratchet-Synchronisation verloren.	*Fatal (Reset nötig)*

4. C# (.NET) Integration & Exception Handling

Der *C#-Wrapper* nutzt ein objektorientiertes Modell mit einer zentralen Client-Klasse. Er abstrahiert die komplexen Padding- und Memory-Operationen der v2.0.0 Engine.

4.1 Implementierungsbeispiel

```
using ProChat;

public void SecureCommunicationExample()
{
    try
    {
        // 1. Initialisierung mit 32-Byte Seed
        byte[] ramSeed = GetSecureEntropy(32);
        var client = new ProChatClient(uid: 1001, seed: ramSeed);

        // 2. Peer hinzufügen
        byte[] sharedKey = GetSharedKeyFromHandshake();
        if (!client.AddPeer(2002, sharedKey))
        {
            throw new InvalidOperationException("Peer-Initialisierung
        }

        // 3. Senden (Die Engine übernimmt automatisch Noise-Padding
        byte[] payload = System.Text.Encoding.UTF8.GetBytes("BrainAI
        byte[] encryptedPacket = client.EncryptMessage(2002, type: 1,

        if (encryptedPacket == null)
        {
            LogError("Verschlüsselungsfehler in der Core Engine (z.B.
        }

        // 4. Empfang und Validierung (inkl. AEAD-Header Check)
        uint senderId;
        byte[] decrypted = client.DecryptPacket(receivedBytes, out se

        if (decrypted == null)
        {
            // Deckt strikte Manipulationen und Buffer-Underflows ab
```

```
            throw new SecurityException("Integritätsprüfung fehlgesch
        }
    }
    catch (ArgumentException ex)
    {
        LogError($"Parameterfehler: {ex.Message}");
    }
}
```

5. Python (High-Performance Scripting)

Der *Python-Wrapper* ist für maximale Flexibilität ausgelegt. Er konvertiert C-Fehlercodes in saubere Python-Rückgabewerte.

5.1 Implementierungsbeispiel

```python
from prochat import ProChat, ProChatStatus

def run_chat_service():
    # Setup Engine
    seed = b'\x12' * 32
    client = ProChat(uid=1001, seed=seed)

    # Peer Setup
    key = b'\xAB' * 32
    if not client.add_peer(peer_uid=2002, key=key):
        print("Fehler: Peer konnte nicht registriert werden.")
        return

    # Sende-Loop (Rest des Pakets wird mit kryptografischem Rauschen
    msg = b"System-Health: OK"
    packet = client.encrypt(target_uid=2002, msg_type=1, message=msg)

    if packet:
        send_to_socket(packet)

    # Empfangs-Logik mit validierter Payload-Länge
    raw_data = socket.receive(1024)
    result = client.decrypt(raw_data)

    if result is None:
        # Fängt PC_ERR_AUTH (-2) Constant-Time Check Ausfälle ab
        handle_security_incident("Manipulation oder Desync erkannt!")
    else:
        sender_id, payload = result
        print(f"Sichere Nachricht von {sender_id}: {payload.decode()}
```

6. Java (JNA / Enterprise Integration)

Die *Java*-Implementierung nutzt eine statische IProChat-Library Instanz und bietet durch die `DecryptedMessage`-Klasse ein Memory-sicheres Datenhandling für Backend-Architekturen.

6.1 Implementierungsbeispiel

```
import com.brainai.prochat.ProChat;

public class SecureGateway {
    public void processIncoming(byte[] packet) {
        ProChat.Client client = new ProChat.Client(1001, mySeed);

        // Entschlüsselungs-Versuch (Liest die exakte Payload-Länge a
        ProChat.DecryptedMessage msg = client.decryptPacket(packet);

        if (msg == null) {
            System.err.println("Kritischer Fehler: Authentifizierung
            auditLog.reportAttack(System.currentTimeMillis(), "MAC_FA
            return;
        }

        System.out.println("Paket authentifiziert von UID: " + msg.se
        processPayload(msg.data);
    }
}
```

7. JavaScript (Node.js / Web-Services)

Der *Node.js* Wrapper nutzt native Buffer für Zero-Copy Operationen und ist ideal für skalierbare Backend-Systeme oder WebAssembly-Edge-Nodes geeignet.

7.1 Implementierungsbeispiel

```
const ProChat = require('./ProChat');

const chat = new ProChat(1001, seedBuffer);

// Senden
const packet = chat.encrypt(2002, 1, "Cloud-Status: Green");
if (!packet) {
    console.error("Verschlüsselungsfehler - Engine-Parameter prüfen (
}

// Empfangen
const result = chat.decrypt(incomingBuffer);
if (!result) {
    // Deckt PC_ERR_AUTH (-2) ab
    console.error("Sicherheitswarnung: Paket-Integrität verletzt!");
} else {
    const { senderUid, payload } = result;
    console.log(`Verifizierte Daten von ${senderUid}:`, payload.toStr
}
```

8. VB.NET (Industrial Systems)

Speziell für industrielle Automatisierungssysteme (OT) bietet der VB.NET-Wrapper volle Kompatibilität und striktes Error-Handling via `IsNot Nothing` Prüfungen, um Anlagen-Stillstände durch Buffer-Overflows zu verhindern.

8.1 Implementierungsbeispiel

```
Public Sub HandleIndustrialMessage(packet As Byte())
    Dim client As New ProChatClient(1001, globalSeed)
    Dim senderUid As UInteger = 0

    ' Entschlüsseln und Validieren (Verhindert fehlerhafte Längen-Aus
    Dim payload = client.DecryptPacket(packet, senderUid)

    If payload Is Nothing Then
        ' Fehlerbehandlung für PC_ERR_AUTH oder PC_ERR_REPLAY
        RaiseSecurityEvent("Integritätsfehler bei Sender " & senderUi
        Exit Sub
    End If

    ' Verarbeitung bei Erfolg
    LogMessage("Sicheres Datenpaket von " & senderUid & " empfangen."
    UpdateMachineState(payload)
End Sub
```

9. Best Practices & Sicherheitsrichtlinien

- **Strict Boundary Control:** Die ProChat Engine v2.0.0 wehrt Payloads > 988 Bytes auf nativer Ebene rigoros ab, um Buffer Overflows zu verhindern. Applikationen müssen diese Limits im Vorfeld (z. B. durch Chunking) respektieren.
- **Sitzungs-Synchronität & KDF:** ProChat nutzt eine hochstrikte KDF-Ratchet (Key Derivation Function). Bei paketbasierten UDP-Übertragungen in gestörten Umgebungen *muss* der Wrapper eine externe Sequenz-Prüfung oder ein Buffering vorschalten, um einen permanenten `PC_ERR_DESYNC` durch Out-of-Order Pakete zu vermeiden.
- **Entropie-Quelle:** Die Engine verlässt sich für das Noise-Padding auf `ProKey`. Stellen Sie sicher, dass das ausführende System über ausreichend Hardware-Entropie verfügt.
- **Speicher-Bereinigung:** In Sprachen mit Garbage Collection (Java, C#, JS) sollten sensitive Buffer nach der Nutzung explizit mit Nullen überschrieben werden, bevor sie für den GC freigegeben werden.

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**